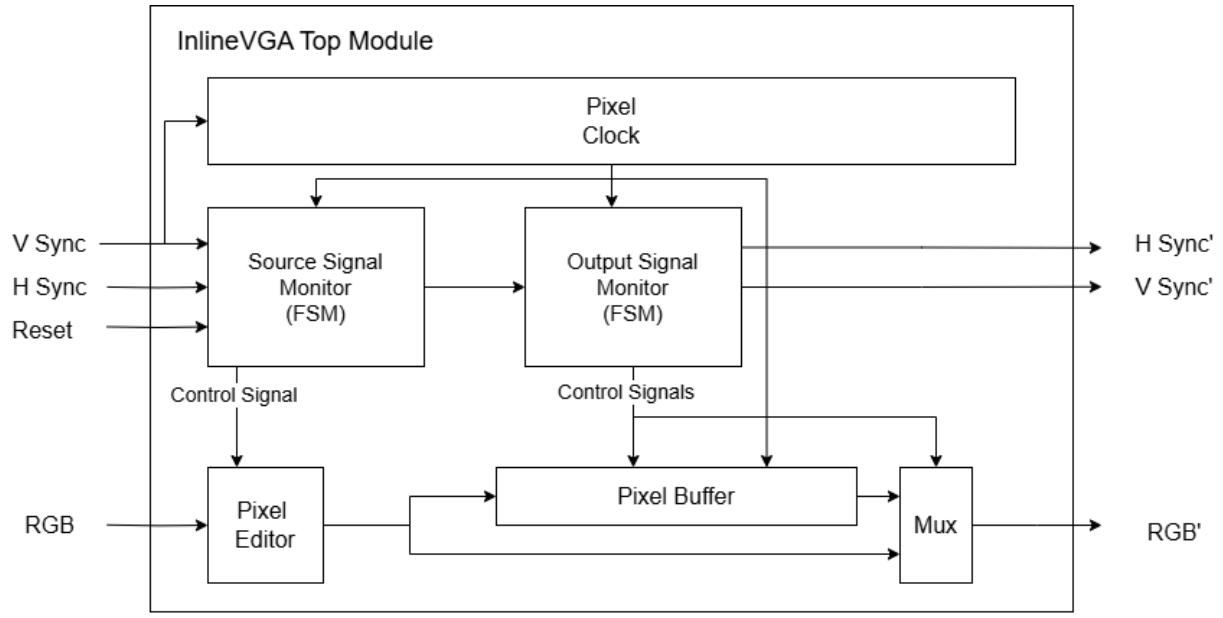
# InlineVGA Design Milestone

ECE 18624 - Special Topics in Chip Design: Open-Source FPGA and ASIC Chip Design

Sean Hyacinthe

## Datapath Block Diagram




## Modules Specifications

### InlineVGA Top Module

This module is the top level module, containing all subcomponents, and will serve the IO interface. The inputs and outputs should match up with the tapeout IO interface requirements.

| Input | Width | Description |
|---|---|---|
| Reset | 1 | Reset for all modules |
| Red | 3 | Red channel data after ADC |
| Green | 3 | Green channel data after ADC |
| Blue | 3 | Blue channel data after ADC |
| H Sync | 1 | Horizontal sync pulse from VGA cable |

| | | |
|---|---|---|
| V Sync | 1 | Vertical sync pulse from VGA cable |
| **Total** | 12 | N/A |

| Output | Width | Description |
|---|---|---|
| Red | 3 | Red channel data before DAC |
| Green | 3 | Green channel data before DAC |
| Blue | 3 | Blue channel data before DAC |
| H Sync | 1 | Horizontal sync pulse from VGA cable |
| V Sync | 1 | Vertical sync pulse from VGA cable |
| Error | 1 | Error if something goes wrong |
| **Total** | 12 | N/A |

## Pixel Clock

The Pixel is responsible for synchronizing when to transition to the next pixel in the row. The pixel clock runs at approximately 25 MHz (based on a 60 Hz refresh rate). The clock will starting when V sync's falling edge (start of vertical sync pulse). By starting on V sync's falling edge it allows the system to synchronize the input signal at a known position on the screen.

| Input | Width | Description |
|---|---|---|
| Reset | 1 | Reset for all modules |
| V Sync | 1 | Vertical sync pulse from VGA cable |

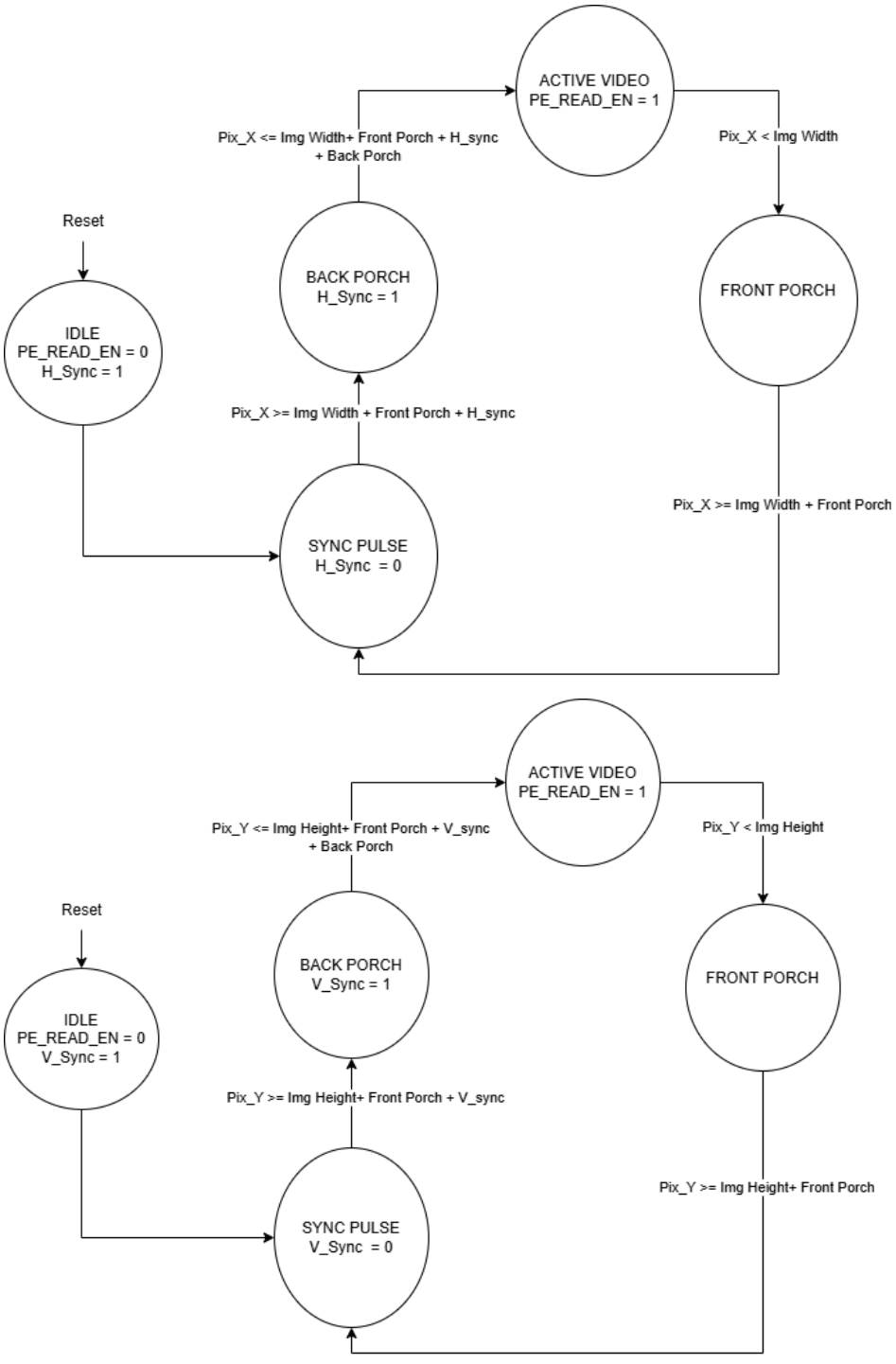
| Output | Width | Description |
|---|---|---|
| Pixel Clock | 1 | Clock for pixel related operations |

## Source Signal Monitor

The Source Signal Monitor is designed to keep track of the source VGA signal and which part of the VGA protocol it is in (Active Video, Front or Back Porch, or Sync Pulse). In addition, it will be responsible for managing the row and column index of the current pixel. This will be implemented as an FSM as shown below (currently excluding control signals). The Source

Monitor will have two control signals (1) disables input into the Pixel Editor and (2) serves as the start signal for the Output Monitor.

## Sync State Machines



| Input | Width | Description |
|---|---|---|
| Reset | 1 | Reset for all modules |
| H Sync | 1 | Horizontal sync pulse from VGA cable |
| V Sync | 1 | Vertical sync pulse from VGA cable |

| Output | Width | Description |
|---|---|---|
| Output Start | 1 | Start signal for the Output Monitor FSM, based on first V Sync |
| Pixel Editor Control | 1 | Control signal to ignore input into the Pixel Editor |

## Output Signal Monitor

The Output Signal Monitor is designed to keep track of the modified VGA signal and which part of the VGA protocol it is in (Active Video, Front or Back Porch, or Sync Pulse). In addition, it will be responsible for managing the row and column index of the current pixel and generating the syncs for the output signal. This will be implemented as an FSM very similar to the Source Monitor FSM. The Output Monitor will have three control signals (1) is a control signal to select if a modified pixel should be passed into the Pixel Buffer or passed directly out, (2) serves as control signals for read and write enables, and (3) will be a select line to choose between the Pixel Buffer output or the direct wire output.

| Input | Width | Description |
|---|---|---|
| Reset | 1 | Reset for all modules |
| H Sync | 1 | Horizontal sync pulse from VGA cable |
| V Sync | 1 | Vertical sync pulse from VGA cable |

| Output | Width | Description |
|---|---|---|
| H Sync’ | 1 | Horizontal sync pulse for outgoing VGA cable |
| V Sync’ | 1 | Vertical sync pulse or outgoing VGA cable |
| Output Path Control | 1 | Select bit to determine if current Pixel Editor output should be passed to Pixel Buffer or a direct wire to output |
| Pixel Buffer Control | 2 | One bit for write and read enable respectively |

| | | |
|---|---|---|
| Final Output Control | 1 | Mux select signal to decide between Buffer value or wire value |

## Pixel Editor

The editor is realistically going to be a wrapper around all the different pixel transformations I have time to implement. If all goes well, I plan to have monochromatic filters for RGB and averaging. The baseline will be swapping RGB values as they come into the ASIC. The inputs of the module will be the digital version of the incoming RGB values and the output will be the Pixel Buffer.

| Input | Width | Description |
|---|---|---|
| Reset | 1 | Reset for all modules |
| RGB | 9 | Digital version of source VGA's RGB values |

| Output | Width | Description |
|---|---|---|
| RGB' | 9 | Digital version of output VGA's RGB values |

## Pixel Buffer

The Pixel Buffer is responsible for queueing RGB values when the outgoing VGA is not consuming pixel data (i.e front and back porch), but the incoming VGA is producing pixel data. The module will take control input from the Output Monitor and pass data to the external output mux. This buffer will also include status for full, which would be a potential indicator of errors.

| Input | Width | Description |
|---|---|---|
| Reset | 1 | Reset for all modules |
| Output Path | 9 | Current Pixel's RGB' values that need to be enqueued |

| Output | Width | Description |
|---|---|---|
| Pixel Buffer Output | 9 | Least recently queued Pixel's RGB' data |

# Initial Test Plan

## Simulation Based

I would like to set up a CocoTB environment where I am able to observe the actual input and output image like we did for the LED strip lab. To enable this, I would need a python script that reads row by row an image and creates the corresponding VGA signal in CocoTB to feed into the system. After getting that set up, the main testing would be ensuring the monitor modules are correctly tracking the state of the VGA signal and asserting the control signals at the proper times. Additionally, it would be good to verify the FIFO behavior and pixel editing operations are functioning as expected. Overall, after the initial setup for reading in image data, testing should be a straightforward process.

## Hardware Based

If time provided, I would like to verify a certain subsystem on the FPGA before moving further along in design. Specifically, I want to test whether the monitor subsystem works by itself. The test would involve connecting the two modules and hard-coding an RGB value to see that I can take the VGA signal and successfully pass it out the VGA monitor.

# Resources

VGA Protocol Information
VGA Timing Specification
TinyVGA Timing Specification